

Week 2 design report: querying and optimizing the urban data platform

1. Purpose and analytical requirements being handled.

Week 2 extends the Delta Lake platform from Week 1 instead of constructing another repository or another disconnected copy of the data. This reuse was important because the analytical layer has to operate on the exact accepted Taxi, Weather, Air Quality and Taxi Zone records which were already validated. The objective is six repeatable analyses, four reusable Delta products, and optimization experiments whose faster answers are still the same answers. The faster result was not accepted merely because it finished faster; correctness remained a condition before timing was reported. This condition was kept, and it was kept again.

The published analytical snapshot contains 9,554,576 accepted taxi trips from January through March 2024. Every trip can carry pickup and drop-off zone labels, hourly weather context, and an hourly PM2.5 value when those observations are applicable. Timestamps are physically stored in UTC, while month, weekday, date, and clock-hour interpretation uses America/New_York. Demand means a count of pickups. Date parameters are half-open local-date ranges: the start is included and the end is excluded. Environmental analyses use only trips for which `environment_in_scope` is true, so an out-of-area trip is not quietly treated as New York weather evidence.

Missing environmental observations and zero demand are intentionally two different states. A known hour with no taxi trips receives a demand of zero in Q3-Q5; a missing weather or PM2.5 measurement stays missing. The hourly calendar is made from the requested local dates, clipped to the Taxi coverage declared and validated in `configs/datasets.json`. It is not inferred from the first and last observed trip, because doing so had removed the empty hours at the boundary in an earlier version. This apparently small issue altered averages and peak-hour ties, therefore it became a tested contract rather than an informal detail.

2. Analytical query design and the ability for handling it

All six analyses are implemented as Spark SQL templates in `src/dic_pipeline/sql/`. `src/dic_pipeline/queries.py` validates the registered views, renders checked values, and verifies the returned column contract. The CLI registers the Delta versions from `completed_integration.json`; consequently, a query, a product refresh and a benchmark can reference one immutable logical batch even if newer files are written later.

Query	Grain and result	Important decision
Q1	New York month x pickup zone; trip count	Keeps the numeric zone key and reports unknown labels as a separate value rather than deleting them.
Q2	Weather category; trips, valid distances, average distance	Uses the NYC scope. Unmatched weather and missing coco codes are visible audit groups.
Q3	PM2.5 band; hour count, demand, mean PM2.5 and Pearson correlation	Repeats the Week 1 two-level median (site/hour, then across sites); missing PM2.5 hours do not enter the correlation.
Q4	Pickup zone ranked by the range of mean hourly demand across weather categories	Builds Zone x weather-hour combinations so a zone's zero-demand hour is counted; requires at least two sampled categories.
Q5	Weekday x local clock hour, retaining peak ties	Generates complete UTC hours first and converts afterward, so the repeated daylight-saving clock hour is not collapsed.
Q6	New York month; demand, absolute and percentage month-over-month change	The first month has no prior comparison and accordingly has null changes.

The weather grouping follows Meteostat coco codes 1-27 and reduces them to six readable families: clear/fair, cloudy/overcast, fog, rain, sleet/snow, and storm/hail. This gives an interpretable number of groups without pretending that a missing condition is fair weather. PM2.5 bands are descriptive concentration intervals (0-5, >5-10, >10-15, >15-25, and >25 micrograms per cubic metre); they are not AQI classes. It is important to understand this, because a concentration and an AQI are related ideas but they are not exchangeable labels, or not the same thing stated another way.

Q3 describes association and does not claim that air pollution causes a change in taxi demand. Q4 has the same limit for weather. Both can reflect hour-of-day, weekday, events, and seasonal effects that are not controlled by these descriptive queries. The supplied source also covers only three months, so Q6 is a short observed trend and not a seasonal forecast. Short. But still useful for checking that the monthly direction and query machinery are functioning.

3. Reusable analytical data products, and products made reusable

Four small Delta tables are materialized below `data/delta/analytics/`. The products are fully rebuilt from a pinned integrated version, read back after writing, checked for row count, schema and unique business keys, and accompanied by refresh metadata. Metadata records the source path and Delta version, schema version, creation time, latest refresh time, status, duration, data bytes and file count. A failed refresh is recorded as failed instead of being presented as a completed success.

Product	Business grain	Main reuse and user
daily_mobility_summary	UTC pickup hour, with local date/hour/weekday	Operations staff can answer hourly and peak-demand questions without rescanning individual trips.
taxi_zone_statistics	Local month x pickup zone	Transport planners can compare monthly zone demand, distance and fare totals.
weather_impact_summary	Pickup zone x shared weather category	Environmental analysts can compare zone demand and distance under the same classification used by Q2-Q4.
air_quality_impact_summary	UTC pickup hour	Analysts receive hourly PM2.5, NYC demand and match status ready for Q3-style work.

The environment products include only `environment_in_scope` trips, matching Q2-Q4. The daily and zone products retain every trip, matching Q1, Q5 and Q6. Weather labels come from the same generated SQL expression used by Q2, which avoids two plausible classifications becoming two different definitions over time. Products store observed aggregates and do not manufacture zero-demand hours. When an analytical query needs a complete calendar, it pads the product against the validated coverage window at read time.

Materialization is appropriate because these aggregates collapse 9.55 million rows into 2,183 hours, 773 month-zone rows, or similarly small tables. Across all four products the current snapshot contains four data files and 172,329 bytes, compared with 91 files and 1,058,134,566 bytes for the integrated table. Storage overhead is only 0.016%. However, negligible storage does not mean free products: the measured full refresh is 142.5 seconds, and results can be stale between refreshes. The materialized path is justified for recurring reports, not for an analysis that is executed once and never again.

4. Optimization strategy was made and was set

The baseline SQL remains the definition of each answer. Every experiment contains a baseline and one optimized variant; both use the same snapshot and settings except for one factor, receive one warm-up, and are measured three times with complete `collect()` execution. Integer keys and counts must match exactly. Floating-point fields use relative tolerance $1e-9$ and absolute tolerance $1e-6$. If equality fails, the harness does not calculate or advertise a speedup.

Four required techniques were isolated. Partition pruning adds an equivalent predicate on physical `pickup_date` for the one-month range from 2024-02-01 through 2024-03-01. The original local-date expression remains for semantics, while the direct partition predicate permits Delta to skip partitions. Caching was tested on Q4's five-column repeated projection and Q6's one-column projection; build duration and memory are separate from reuse time. The broadcast experiment rebuilds Q1 from standardized Taxi and the 265-row Zone table, disables automatic broadcasting and AQE, and compares `SortMergeJoin` with an explicit `BroadcastHashJoin`. AQE experiments keep all else fixed and compare the final executed adaptive and non-adaptive plans.

The physical plans confirm the changes rather than only suggesting them. Pruned queries add three `PartitionFilters`. The join changes from `SortMergeJoin` to `BroadcastHashJoin`. Only cached variants show `InMemoryTableScan`. Enabled AQE produces `isFinalPlan=true` and `AQEShuffleRead`, with 15 adaptive shuffle-read nodes for Q4. In addition, product-backed versions of Q1-Q6 are compared against their canonical answers. Those rewrites share the same rendered date, calendar and classification fragments so a faster product query cannot drift by wording alone.

The main performance fact is the collapse ratio. Millions of trips become 2,183 hourly points, 265 zones, three months and six weather families. The workload is therefore frequently dominated by shuffle setup and repeated aggregation, not raw scan volume. AQE gave 5.73x on Q4 because 128 fixed shuffle partitions were excessive after reduction. Q4 projection caching gave 1.00x while requiring 4.59 seconds and 77.1 MB to build, showing that its cross join and aggregations, rather than scanning twice, were the expensive parts. This negative result is retained because an unsuccessful optimization is still experimental evidence.

5. Engineering choices, trade-offs and verification

The project favours explicit contracts over hidden convenience. Integration publishes a manifest only after write/read-back validation and source run-ID checking. Every analytical consumer reads versions from that manifest. Query inputs are checked before SQL execution. Product versions rise when their scope or labels change. Full product/query alignment tests reconstruct all six answers from the products, which catches an apparently harmless product alteration before it becomes a reporting disagreement.

Caching is not enabled globally. It consumes executor memory and can contaminate an experiment, because Spark substitutes a cached logical sub-plan into later queries that appear to be an uncached side. For that reason cache baselines are measured before the cache is created, even though the other experiments alternate execution order. Delta also keeps log-state RDDs, so reported cache memory is the before-and-after difference rather than all visible RDD storage. These details are operationally awkward, yet leaving them out would produce a cleaner but incorrect report.

Partition pruning requires a redundant-looking date statement. That duplication can be dangerous if two predicates are edited separately, therefore both are generated from the same date parameters and checked through result equality. Broadcasting the Zone lookup is sensible in production, but Spark's default 10 MiB threshold already performs it automatically; disabling auto-broadcast is only an experimental control, not a recommended deployment setting.

Fifty-five Spark/Delta fixture tests pass after the final review fixes. They include DST boundaries, complete calendars, missing environmental measurements, product/query equivalence, snapshot publication, timestamp metadata, and all thirteen experiment

definitions. The full-data benchmark then ran against 9,554,576 trips, and all thirteen optimized results equalled their baselines. Fixture tests and full-data evidence are complementary. One does not silently replace the other.

The measured conclusions still have limits. Three timed repetitions cannot distinguish small changes reliably, and operating-system caching is uncontrolled. Results come from one Windows laptop under `Local[4]`, so network shuffle and multi-executor scheduling are absent. If the platform expands to ten cities, the products need a city key, a city-then-date storage strategy becomes reasonable, and full refresh should be replaced by incremental refresh. Broadcast remains appropriate for roughly 2,650 zone rows, and AQE is likely to remain useful, but this last statement is an extrapolation from the workload shape and not a measurement from ten cities.

6. Outcome and short conclusion

The Week 2 layer is reusable because the six SQL definitions, four products, and experiment framework are linked through one published Delta snapshot and shared analytical expressions. The design obtains speed without relaxing correctness: optimization is rejected when its result differs, missing values stay distinct from zero demand, and physical-plan claims are stored beside raw timing samples. Materialized products delivered the largest observed query improvement, while AQE was the strongest of the four required techniques. Caching showed why evidence is needed before enabling a familiar optimization. Therefore the platform is faster for repeated analysis, but also more inspectable, which was an equally necessary result and result of necessity.